



Eventos Evaluativos #3 y #4

API Sports League

Seguimiento del proyecto + Implementación de Alineaciones (Match Lineup)

Valor total: 40% (20% cada evento)

Modalidad: Individual | Fecha límite: Viernes 22 de mayo hasta las 23:59

EVENTO EVALUATIVO #3 — Seguimiento del Proyecto (20%)

1. Descripción

Este evento evalúa que el estudiante esté al día con todas las fases del proyecto API Liga Deportiva desarrolladas en clase. No se requiere implementar nada nuevo; se verifica que el proyecto compile, ejecute correctamente, y que todas las funcionalidades vistas estén implementadas y funcionando.

2. Fases que se evalúan

Fase	Nombre	Qué se revisa
1	Team CRUD	Entidad Team, GenericRepository, TeamService, DTOs, Controller
2	Player con FK	Entidad Player con FK a Team, Enum PlayerPosition, validaciones
3	Referee + Tournament N:M	Referee, Tournament, TournamentTeam (N:M), inscripción de equipos
4	Match + Máquina de Estados	Entidad Match con múltiples FK, estados (Scheduled→InProgress→Finished→Cancelled)
5	Result, Goals, Cards	MatchResult (1:1), Goal, Card, MatchEventService, validaciones cruzadas
5.1	DataSeeder	Clase DataSeeder en DataAccess/Seeders con 20 equipos BetPlay, jugadores, árbitros, torneo
6	Standings + Stats	Tabla de posiciones y estadísticas calculadas con LINQ (no almacenadas)

3. Criterios de evaluación

#	Criterio	Descripción
1	Compilación y ejecución	El proyecto compila sin errores y la API inicia correctamente
2	Swagger funcional	Todos los endpoints visibles y probables desde Swagger
3	CRUD completo	Teams, Players, Referees, Tournaments funcionan (crear, listar, editar, eliminar)
4	Partidos y eventos	Crear partidos, registrar goles, tarjetas y resultados correctamente
5	DataSeeder	Al ejecutar la API con BD vacía, se pueblan datos automáticamente
6	Standings y Stats	Endpoints de posiciones y estadísticas retornan datos calculados correctamente

Obtendrás una nota de 5.0 con solo estar al día hasta la fase 6

4. Forma de entrega

A través de la URL del repo en GitHub. No se requiere hacer video ya que en el Evento Evaluativo #4 que se explicará a continuación se podrá evidenciar que estás al día con todas las fases hasta la #6.

❏ Importante

Si el proyecto no compila o no ejecuta, la nota máxima del Evento #3 será 1.0. Es responsabilidad del estudiante tener el proyecto funcional antes de la revisión.

EVENTO EVALUATIVO #4 — Alineaciones de Partido (20%)

1. Descripción

Implementar la funcionalidad de Alineaciones de Partido (Match Lineup) en la API Liga Deportiva. Esta funcionalidad permite registrar los jugadores convocados para un partido, indicando cuáles son titulares y cuáles suplentes. Es una extensión del proyecto existente que aplica los patrones aprendidos en las Fases 1 a 5.

2. Lo que se debe implementar

2.1 Entidad MatchLineup

Crear una nueva entidad que represente la convocatoria de un jugador a un partido:

Propiedades requeridas:

Id	(int)	→ Primary Key
MatchId	(int)	→ FK a Match
PlayerId	(int)	→ FK a Player
IsStarter	(bool)	→ true = Titular, false = Suplente
Position	(string)	→ Posición asignada para este partido (ej: "GK", "CB", "CDM", "ST")

Relaciones de navegación:

Match	(Match)	→ Navegación al partido
Player	(Player)	→ Navegación al jugador

2.2 Ubicación de los archivos

Seguir la misma estructura de capas del proyecto

2.3 Validaciones de negocio (obligatorias)

El Service debe implementar las siguientes validaciones:

#	Validación	Mensaje de error sugerido
V1	El partido debe existir	"No se encontró el partido con ID {matchId}"
V2	El jugador debe existir	"No se encontró el jugador con ID {playerId}"
V3	El jugador debe pertenecer al HomeTeam o AwayTeam del partido	"El jugador no pertenece a ninguno de los equipos del partido"
V4	El jugador no puede estar registrado dos veces en la misma alineación	"El jugador ya está registrado en la alineación de este partido"
V5	Máximo 11 titulares por equipo por partido	"El equipo ya tiene 11 titulares registrados en este partido"
V6	El partido debe estar en estado Scheduled	"Solo se pueden registrar alineaciones en partidos Scheduled"

□ Sobre las validaciones

Estas validaciones siguen el mismo patrón que aprendimos en la Fase 5 con Goals y Cards. V3 es idéntica a la validación que hace MatchValidationHelper. Se puede reutilizar el Helper o duplicar la lógica en el nuevo Service.

2.4 Endpoints a implementar

Método	Endpoint	Descripción
POST	/api/match/{matchId}/lineup	Agregar un jugador a la alineación
GET	/api/match/{matchId}/lineup	Obtener la alineación completa del partido
GET	/api/match/{matchId}/lineup/team/{teamId}	Obtener alineación de un equipo específico
DELETE	/api/match/{matchId}/lineup/{id}	Eliminar un jugador de la alineación

2.5 DTOs

Los DTOs deben separar la entrada (lo que envía el cliente) de la salida (lo que retorna la API):

```
// CreateMatchLineupDto (entrada)
PlayerId      (int)
IsStarter      (bool)
Position       (string) // Ejemplo: "GK", "CB", "CDM", "CAM", "ST"

// MatchLineupDto (salida)
Id             (int)
MatchId        (int)
PlayerId       (int)
PlayerName     (string) // Nombre completo del jugador
TeamName       (string) // Nombre del equipo del jugador
IsStarter      (bool)
Position       (string)
```

2.6 Configuración en DbContext

Igual como hemos venido trabajando con las anteriores entidades en el DbContext.

□ Índice único compuesto

El índice único `HasIndex(ml => new { ml.MatchId, ml.PlayerId }).IsUnique()` es la misma técnica que usamos en `TournamentTeam` (Fase 3). Garantiza que la BD no permita registrar el mismo jugador dos veces en el mismo partido, incluso si falla la validación del Service.

No olvidar hacer las migrations.

3. Escenarios de prueba en Swagger

Se espera que el estudiante demuestre los siguientes escenarios funcionando:

#	Escenario	Resultado	Código HTTP
1	POST un jugador titular del HomeTeam a un partido Scheduled	✓ Creado	201
2	POST un jugador suplente del AwayTeam al mismo partido	✓ Creado	201
3	POST el mismo jugador otra vez al mismo partido	✗ Duplicado	409
4	POST un jugador que no pertenece a ninguno de los dos equipos	✗ No pertenece	409
5	POST un 12° titular para un equipo (ya hay 11)	✗ Máx 11	409
6	POST a un partido en estado Finished	✗ Estado inválido	409
7	GET la alineación completa del partido	✓ Lista	200

#	Escenario	Resultado	Código HTTP
8	GET la alineación filtrada por equipo	✓ Filtrada	200
9	DELETE un jugador de la alineación	✓ Eliminado	204
10	DELETE con ID inexistente	✗ No encontrado	404

4. Rúbrica de evaluación

#	Criterio	Descripción	Puntos
1	Entidad + DbContext	Entidad MatchLineup con propiedades correctas, Fluent API, índice único compuesto, migración aplicada	1.0
2	Repository	IMatchLineupRepository + MatchLineupRepository con métodos específicos (GetByMatch, GetByMatchAndTeam, ExistsByMatchAndPlayer)	0.8
3	Service + Validaciones	MatchLineupService con las 6 validaciones obligatorias (V1-V6) implementadas correctamente	1.2
4	DTOs + AutoMapper	CreateMatchLineupDto, MatchLineupDto, perfil de AutoMapper con mapeo de PlayerName y TeamName	0.5
5	Controller + Endpoints	4 endpoints REST funcionando, respuestas HTTP correctas (201, 200, 204, 404, 409)	1.0
6	Demo en Swagger	Los 10 escenarios de prueba funcionan correctamente en vivo	0.5

Total: 5.0 puntos (equivale al 20% de la nota final)

5. Patrones de referencia

Todo lo que necesitas para este evento ya lo aprendiste en fases anteriores. Esta tabla te indica dónde buscar cada patrón:

Lo que necesitas hacer	Fase donde se aprendió	Ejemplo en el proyecto
Crear entidad con FK	Fase 2	Player → Team (FK)

Lo que necesitas hacer	Fase donde se aprendió	Ejemplo en el proyecto
Índice único compuesto (N:M)	Fase 3	TournamentTeam
Validar jugador pertenece al equipo del partido	Fase 5	MatchValidationHelper
Validar estado del partido	Fase 4	MatchService (máquina de estados)
Contar registros con LINQ	Fase 6	Standings (Count, Sum)
Endpoints anidados bajo /match/{id}/	Fase 5	/api/match/{id}/goals
DTOs con AutoMapper	Fase 1-5	Todos los DTOs del proyecto
Repositorio específico	Fase 3-5	MatchResultRepository

📌 Consejo

Antes de empezar, revisa cómo están implementados Goal y Card en la Fase 5. La entidad MatchLineup sigue exactamente el mismo patrón: una entidad con FK a Match y FK a Player, validaciones cruzadas, y endpoints anidados bajo /api/match/{matchId}/lineup.

6. Forma de entrega

- El estudiante debe entregar la URL del repositorio en GitHub con el código fuente completo del proyecto, incluyendo la funcionalidad de Alineaciones (MatchLineup). El proyecto debe compilar y ejecutar sin errores.



Video OBLIGATORIO

El video de sustentación es un entregable obligatorio. Sin video, la calificación será de 0.0, el video es requisito fundamental para demostrar el conocimiento, un repositorio solo con código no me dice nada, cualquiera lo puede hacer, el video no. Por eso es fundamental la grabación del video.

- Sustentación en video:** El estudiante debe grabar un video de sustentación (máximo 10 minutos) en el que demuestre la funcionalidad implementada y explique su código. El video debe contener lo siguiente:
 - Demostración de escenarios de prueba: Seleccionar y ejecutar desde Swagger mínimo 3 escenarios positivos (✓) y 2 escenarios negativos (✗) de los 10 propuestos en el punto 3 de este documento. El estudiante elige cuáles demostrar.
 - Explicación del código: No se requiere explicar la arquitectura ni los patrones aplicados (eso ya se evaluó en eventos anteriores). Lo que se evalúa es que el

estudiante explique con propiedad la lógica de negocio que implementó: qué hace cada método, cómo funcionan las validaciones, y por qué se implementaron de esa manera. Se debe demostrar dominio de lo que se escribió, no solo que funciona.

❑ Regla de sustentación

El estudiante debe poder explicar cada línea de código que implementó. Si no puede explicar una validación o un endpoint, ese criterio se califica con 0. Copiar código sin entenderlo no es válido.

7. Preguntas frecuentes

¿Puedo reutilizar el MatchValidationHelper?

Sí, es lo recomendado. Si lo inyectas en tu MatchLineupService puedes reusar ValidateMatchForEventAsync y ValidatePlayerInMatchAsync directamente.

¿Los suplentes también cuentan para la validación V5 del máximo 11?

No. La validación V5 solo aplica para titulares (IsStarter = true). Se pueden registrar suplentes sin límite.

¿Qué valores pongo en el campo Position?

Es un string libre. Puede ser abreviaciones como "GK", "CB", "LB", "RB", "CDM", "CM", "CAM", "LW", "RW", "ST". El profesor no evaluará qué valor se ponga, solo que el campo exista y se guarde correctamente.